DBinary
新知答主

科学季 2025 ›

1232 人赞同了该回答 ›

字符编码，我愿称之为计算机历屎上的屎山王中王。

其支撑起了无数次谁都不服谁的争论。

几乎把所有主流操作系统都坑的不要不要的，并且至今仍然遗留着一大堆乱七八糟的问题。

并且最重要的是至今没有一个较为完美的解决方案。

我们先回到几十年前的某一天,作为一个根正苗红的阿美莉卡计算机攻城狮,你认为存储一个"字母",用几个bit合适?

已知我们有26个英文字母,我们算大小写翻个倍儿,加上数字标点,怎么的128种编码情况也够了吧,加上1个位作为拓展,一字节=8位,简直特么的完美.

可以说ASCII字符集✦终结了1byte=6位,7位,8位,9位这群魔乱舞的时代,但你别急,我们的故事才刚刚开始.

随着计算机的流行和发展,肯定不只有阿美莉卡在用啊,就说我们中文,<<汉语大字典>>收录的数量就有5万个,哪怕我们凑活凑活,只用常用汉字,那也有3000个左右,8bit能表示的范围就256个,这显然不够啊

你会说,不要慌,8bit不还给你留了一个扩展位么,我们可以这样,第一个字节如果高位是1,那么后面那个就是扩展位,所以在1980年中国国家总局颁布了第一个中文编码GB2312-80,不过这个编码的覆盖范围不够,所以后面又颁布了GBK编码✦,

简单来说如果第一个字节的范围是0x81–0xFE,那么它就是一个汉字,你需要和第二个字节(范围0x40–0xFE）去凑够整个汉字的编码.

很完美是不是?这是一个神奇的编码,具体怎么神奇我们稍候再说。

中文问题解决了，但是又有一个大问题，比如90年的后即十几二十年年，windows是毫无疑问的操作系统一哥，没有之一，但显然windows不能只给我们中国用，它得挑一个别的编码，大到能够覆盖大部分国家的绝大多数语言。

这问题就大了，选啥好呢？

1991年Unicode✦ 1.0刚刚诞生,当时的UCS-2(也就是后来的UTF16✦)支持65536个码位,这就像IPv4地址被设计成32位被认为完全够用一样,当年他们也认为UCS-2足够覆盖所有的文字了。

那么，现在第一个问题来了，现在我们在网上聊天，在经过了一系列深入的交流后，我们决定对《金瓶梅》这部小说进行严肃的学术探讨，所以我把这本书的电子版txt发给了你。

那么它到底是GBK编码还是UTF16呢（UCS-2后来的名称，后文为了方便我们都用UTF16）。

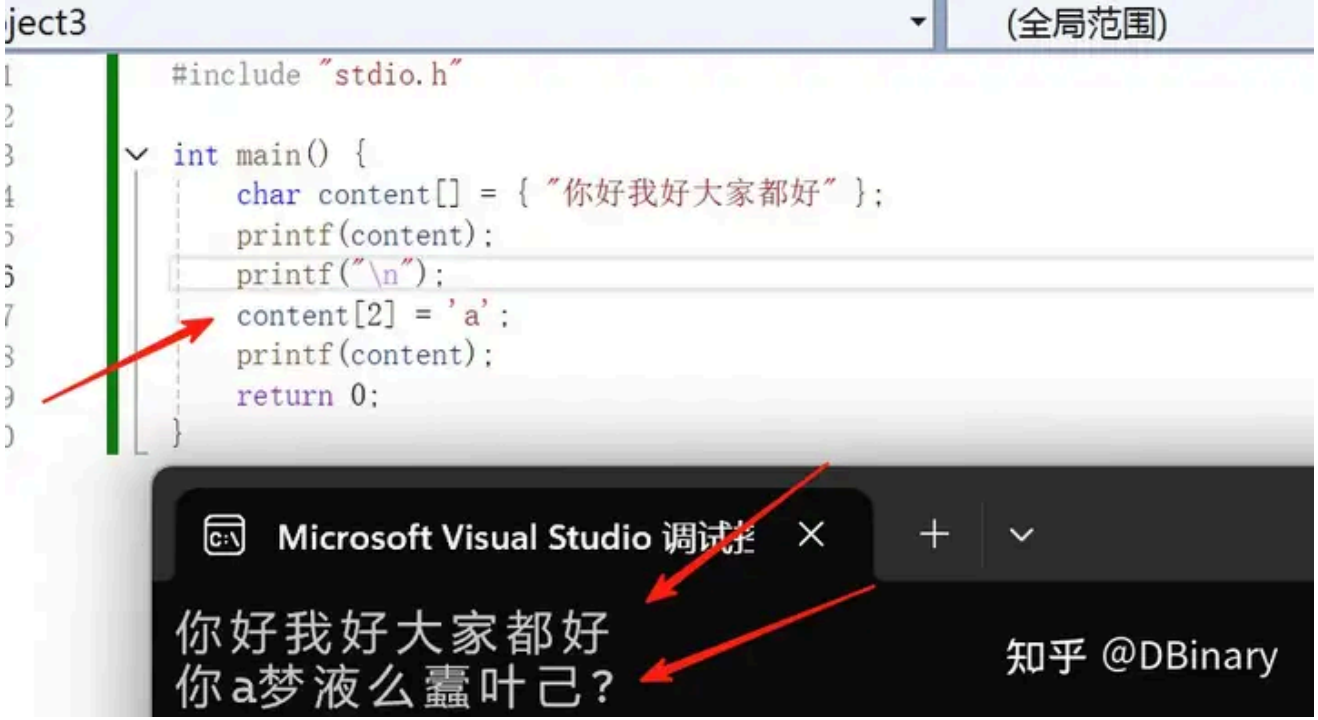
然后我们都发现了一个问题，我们竟然无法通过txt这个文件的任何数据，知道它究竟是什么编码，所以，我们只能一个一个试过去，先用UTF16打开，哎呀，乱码，再用GBK格式打开，哦，正确显示了，所以它是GBK编码

但是，我们人还可以通过视觉手段判断一下到底是什么编码，但很多的文本处理程序可就要了老命了，因为文本的解析完全依赖编码，所以要么设置一个默认编码，要么让用户选一个。

要知道，很多的文员连键盘都用的不利索，你和他们说什么文本编码，那不为难我胖虎么？

现在，第一个坑还没解决，我们回到“GBK”是一个神奇的编码这个问题上来，为什么神奇呢，第一个它是非国际标准编码，所以GBK编码，几乎不具备通用性，特别像早期的Linux系统发行版，根本就不鸟你，但最关键的问题是，GBK编码几乎没有纠错能力和快速索引能力

非常容易出现那种错了一个字节后面全错的情况（下面的程序故意误码1字节，导致后面的文本全错）：



而快速索引指的是比如你要如何定位第1000个汉字呢，只要文本中有一个数字英文或半角标点什么的，你不得不从头解析整个字符串，就比如我们有一本长篇小说，我们想找到书里的第100000个汉字，我们不得不从头开始解析文本，这就导致了程序的文本处理效率极差。

更坑爹的是GBK编码第二个字节的高位不一定是1，这就导致了其可能与ASCII码混叠在一起，假如你在写一个编译器需要非常小心的处理字符串编码，但某天你没有正确的识别（回到上面那个编码识别问题）代码编码，非常可能错误的把汉字的一个字节识别成ASCII字符，那你这个编译就直接炸了。

所以即使到今天，GBK编码仍然是一个“非常难搞”的编码。

你是不是想说那我们不用不就得了，我们完全采用Unicode的几种编码。

别急，Windows也后院起火了，今天我们知道了UTF16，这特么完全就是一个神坑啊。

首先，UTF16是一个多字节编码，那么问题来了，它到底是大端序还是小端序的呢？傻了吧，UTF16这种编码内部还分家了，还直接拉上了另一个计算机科学里另一个“先有鸡还是先有蛋”的

神坑问题里------到底是特么的大端好，还是小端好。

当然这不是什么事，真正坑爹的是，今天我们知道UTF16完全不够用，后来Unicode标准扩展到超过100 万个码位。UTF16 为了兼容，发明了代理对这种神奇的玩意儿，这就导致了UTF16不再是定长编码而需要变长解析。

好家伙，合着UTF16是在统一编码快速索引一个问题都没解决，还在屎山上继续拉了一坨大的。

所以到今天，你可以看到GBK和UTF16在windows仍然坚挺，

属实是自己装的逼，再难受也得装下去。

但问题还是没有解决啊，要知道Unicode今天可不止一种编码，今天有UTF8+ UTF16 UTF32+，其中多字节编码还有细分大小端（LE BE）问题。

兜兜转转，结果发现特么我们还是一个问题都没解决，一个文本文件过来，它到底是哪种编码。

这个时候微软又出来说，要不我们在文本文件最前面几个字节里标识一下这个文本文件到底用什么编码吧，这也就是我们今天说的BOM头

不同编码下，BOM 表示的字节序列不同：

编码方式	BOM字节序列（十六进制）	说明
UTF-8	**EF BB BF**	仅标识 UTF-8，无大小端之分
UTF-16 LE	**FF FE**	UTF-16 小端（低字节在前）
UTF-16 BE	**FE FF**	UTF-16 大端（高字节在前）
UTF-32 LE	**FF FE 00 00**	UTF-32 小端
UTF-32 BE	**00 00 FE FF**	UTF-32 大端

（LE = Little Endian 小端，BE = Big Endian 大端）

知乎 @DBinary

可惜，当时已经不是微软一家独大他说的算了，显然是上个逼他装的太失败了，比如Linux Unix 和GNU一众表示，微软你特么就一个行业毒瘤，你说要支持就支持，那我不特么很没面子。

何况作为文本文件，前面还要塞一些东西进去，这会破坏ASCII特性，那这个文本文件还是文本文件么，那不成了四不像，不然你看我HTTP协议，说好的超文本传输协议，难不成也要加个BOM头，你丫的纯属脱裤子放屁，所以哪怕时至今日，BOM头仍然在很多需要进行文本分析的软件和平台的支持受阻。

并且，即使Unicode编码的多种编码格式，与其说每种都解决了一些问题，不如说每种也都有严重的问题。

例如作为目前最广泛的UTF8现在各平台的支持度最好，但是仍然存在索引缓慢的问题，也就是之前说的你要定位某一个汉字，你需要从头开始解码整段文本，效率很低。而UTF32虽然能完美覆盖所有编码，但对于最常用的纯ASCII字符文本，虽然解决了动态索引速度问题，但其内存占用量直接翻了四倍。

所以目前主流的方案是，编码大多采用UTF8形式，解析完成后内存以32位Unicode形式存储。

除此之外，如何判断文本字符串结束，也常常打起来。

保守派认为文本以0结尾是一个优雅的实现。

激进派认为文本应该是抽象数据结构，应该在开头就标明文本长度（甚至编码）。

保守派认为，激进派这种设计结构会破坏数据的纯粹性，比如C语言中严格意义上只有以字符编码的数组而不是什么字符串结构，你加那么多莫名其妙的东西进去，那你说长度应该是几字节，到底用大端编码还是小端，如果长度数据也被破坏了咋办，char txt[]="abc",txt[0]到底是'a'还是什么别的神奇的东西?很多设计就是没事找事自己抽象把自己抽出事了.

激进派则认为保守派这种0结尾的方式造就了无数溢出安全问题,并且每次统计字符串长度都要从头扫描一遍,你不累么?你就是老登,看不得新登的好.

时至今日,这场屎山大战仍然在继续,至于有没有从失败中吸取经验教训.

反正至少目前为止,吸取了经验也没成功过.

额呵~

编辑于 2025-09-05 16:50 · 福建